

Practical: Parallel Depth First Search using OpenMP

Laboratory Practice V – Computer Engineering

1 Aim

Design and implement Parallel Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for DFS.

1.1 Objective

Students should be able to perform Parallel Depth First Search based on existing algorithms using OpenMP.

2 Theory

2.1 What is DFS?

DFS (Depth-First Search) is a graph traversal algorithm that explores as **deep as possible** along each branch before backtracking. Unlike BFS which goes level-by-level, DFS dives deep into one path first.

It uses a **Stack** data structure (LIFO – Last In First Out) to keep track of nodes. DFS can be implemented using:

- **Recursive approach** – uses the system call stack (simpler but may cause stack overflow for large graphs)
- **Iterative approach** – uses an explicit stack (preferred for larger graphs)

A standard DFS puts each vertex into one of two categories:

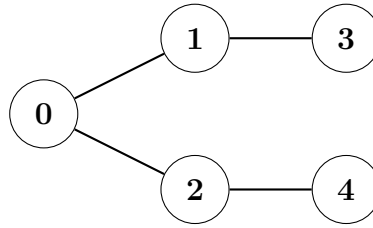
1. **Visited**
2. **Not Visited**

2.2 DFS Algorithm Steps

1. Create a stack. Choose a starting vertex and push it onto the stack.
2. Pop the top vertex from the stack. If not visited, mark it as visited and print it.
3. Push all unvisited adjacent vertices of the current vertex onto the stack.
4. Repeat steps 2–3 until the stack is empty.

2.3 DFS Example

Consider the following undirected graph:



DFS Traversal from Node 0:

Step	Stack	Action	Output
1	[0]	Push 0	–
2	[] → Push 1,2	Pop 0, Visit 0	0
3	[1] → Push 3	Pop 2, Visit 2	0, 2
4	[1]	Pop 4, Visit 4	0, 2, 4
5	[] → Push 3	Pop 1, Visit 1	0, 2, 4, 1
6	[]	Pop 3, Visit 3	0, 2, 4, 1, 3

DFS Output: 0 → 2 → 4 → 1 → 3

Note: DFS output depends on the order in which adjacent nodes are pushed. The key property is that it goes *deep* first before backtracking.

2.4 DFS vs BFS

Feature	DFS	BFS
Data Structure	Stack (LIFO)	Queue (FIFO)
Traversal Order	Depth-wise (deep first)	Level-wise (breadth first)
Shortest Path	Not guaranteed	Yes (unweighted graphs)
Memory Usage	Less (only current path)	More (entire level stored)
Implementation	Recursive or Iterative	Iterative (queue-based)
Backtracking	Yes	No
Parallelism	Task-based (harder)	Level-based (easier)

2.5 Applications of DFS

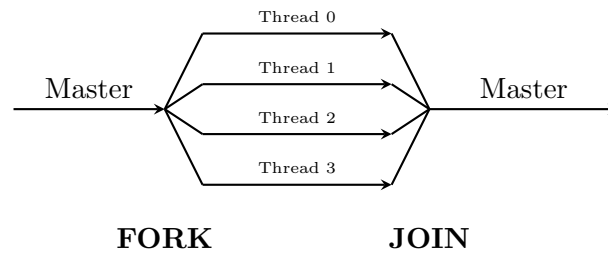
- Cycle detection in graphs
- Topological sorting
- Finding connected components
- Solving mazes and puzzles
- Path finding in games
- Detecting bridges and articulation points

2.6 Concept of OpenMP

OpenMP (Open Multi-Processing) is an API for shared-memory parallel programming in C, C++, and Fortran. Key concepts:

- Uses compiler directives (`#pragma`) to parallelize code.
- Follows a **Fork-Join** model of execution.
- Supports loop parallelism, task parallelism, and synchronization.

2.6.1 Fork-Join Model



2.6.2 Important OpenMP Directives

Directive	Purpose
<code>#pragma omp parallel</code>	Creates multiple threads
<code>#pragma omp parallel for</code>	Divides loop iterations among threads
<code>#pragma omp critical</code>	Only one thread enters at a time (mutex)
<code>#pragma omp single</code>	Only one thread executes the block
<code>#pragma omp barrier</code>	Synchronization – wait for all threads
<code>#pragma omp task</code>	Creates an independent task

2.7 How Parallel DFS Works

Parallel DFS works by distributing the exploration of adjacent nodes among multiple threads:

1. Start with the source node on the stack.
2. Pop a node from the stack. If not visited, mark it visited and print it.
3. Use `#pragma omp parallel for` to push unvisited adjacent nodes onto the stack **in parallel**.
4. Each thread checks a different adjacent node simultaneously.
5. Repeat until the stack is empty.

Key Difference from Sequential DFS: In sequential DFS, adjacent nodes are checked one by one. In parallel DFS, multiple adjacent nodes are checked *simultaneously* by different threads, speeding up the exploration.

3 Code

Listing 1: Parallel DFS using OpenMP

```

1 #include <iostream>
2 #include <vector>
3 #include <stack>
4 #include <omp.h>
5
6 using namespace std;
7
8 const int MAX = 100000;
9 vector<int> graph[MAX];    // Adjacency list representation
10 bool visited[MAX];        // Track visited nodes
11
12 void dfs(int node) {
13     stack<int> s;
14     s.push(node);         // Push starting node
15
16     while (!s.empty()) {
17         int curr_node = s.top();
18
19         if (!visited[curr_node]) {
20             visited[curr_node] = true;    // Mark as visited
21
22             s.pop();
23             cout << curr_node << " ";    // Print current node
24
25             // PARALLEL: Push adjacent unvisited nodes
26             #pragma omp parallel for
27             for (int i = 0; i < graph[curr_node].size(); i++) {
28                 int adj_node = graph[curr_node][i];
29                 if (!visited[adj_node]) {
30                     s.push(adj_node);
31                 }
32             }
33         }
34     }
35 }
36
37 int main() {
38     int n, m, start_node;
39     cout << "Enter no. of Node, no. of Edges and Starting Node:\n";
40     cin >> n >> m >> start_node;
41
42     cout << "Enter pair of edges:\n";
43     for (int i = 0; i < m; i++) {
44         int u, v;
45         cin >> u >> v;
46         graph[u].push_back(v);    // Add edge u -> v
47         graph[v].push_back(u);    // Add edge v -> u (undirected)
48     }
49
50     // Initialize visited array in parallel

```

```

51  #pragma omp parallel for
52  for (int i = 0; i < n; i++) {
53      visited[i] = false;
54  }
55
56  dfs(start_node);
57
58  return 0;
59 }

```

4 Code Explanation

4.1 Data Structures Used

Variable	Type	Purpose
graph[MAX]	vector<int>[]	Adjacency list – stores neighbors of each node
visited[MAX]	bool[]	Tracks which nodes have been visited
s	stack<int>	Stack for iterative DFS traversal

4.2 Graph Representation – Adjacency List

The graph is stored as an **adjacency list**. For each node, a vector stores its neighbors.

Example: For edges 0-1, 0-2, 2-4:

Node	Neighbors
0	[1, 2]
1	[0]
2	[0, 4]
4	[2]

Since the graph is **undirected**, each edge is added in both directions:

```

graph[u].push_back(v); // u -> v
graph[v].push_back(u); // v -> u

```

4.3 DFS Function – Step by Step

1. **Push start node** onto the stack.
2. **Loop** while stack is not empty:
 - (a) Get the top node (`s.top()`).
 - (b) If **not visited**: mark visited, pop it, print it.
 - (c) Use `#pragma omp parallel for` to push all unvisited neighbors **in parallel**.
3. Stack empty $\Rightarrow$ DFS complete.

4.4 OpenMP in This Code

1. Parallel neighbor exploration:

```
#pragma omp parallel for
for (int i = 0; i < graph[curr_node].size(); i++) {
    int adj_node = graph[curr_node][i];
    if (!visited[adj_node]) {
        s.push(adj_node); // Multiple threads push simultaneously
    }
}
```

This divides the adjacent nodes among threads – each thread checks and pushes different neighbors.

2. Parallel initialization:

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    visited[i] = false; // Each thread initializes different elements
}
```

4.5 What is a Race Condition?

A **race condition** occurs when multiple threads access shared data simultaneously and the result depends on the timing of thread execution.

In this code: Multiple threads may push to the stack simultaneously. The **stack** is not thread-safe by default. In a production environment, `#pragma omp critical` should protect stack operations:

```
#pragma omp critical
{
    s.push(adj_node); // Only one thread pushes at a time
}
```

How to avoid race conditions in OpenMP:

- `#pragma omp critical` – mutual exclusion (one thread at a time)
- `#pragma omp atomic` – atomic operations on single variables
- Use thread-local variables where possible
- `#pragma omp barrier` – synchronization points

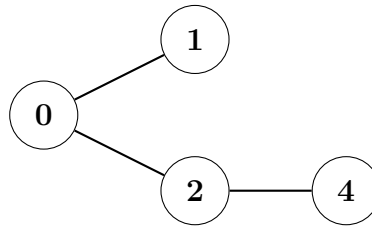
5 Output

5.1 Example 1: Simple Graph

Input:

```
Enter no. of Node, no. of Edges and Starting Node:
4 3 0
Enter pair of edges:
0 1
0 2
2 4
```

Graph:



DFS Traversal (from node 0):

Step	Stack	Action	Output
1	[0]	Push start node 0	–
2	[1, 2]	Pop 0, Visit 0, Push neighbors 1,2	0
3	[1, 4]	Pop 2, Visit 2, Push neighbor 4	0, 2
4	[1]	Pop 4, Visit 4, No unvisited neighbors	0, 2, 4
5	[3]	Pop 1, Visit 1, No unvisited neighbors	0, 2, 4, 1
6	[]	Stack empty – DONE	0, 2, 4, 1

Output: 0 2 4 1

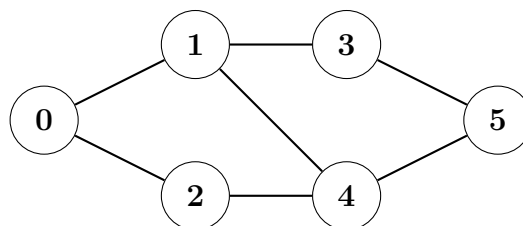
5.2 Example 2: Larger Graph

Input:

```

Enter no. of Node, no. of Edges and Starting Node:
6 7 0
Enter pair of edges:
0 1
0 2
1 3
1 4
2 4
3 5
4 5
  
```

Graph:



DFS Output (from node 0): 0 2 4 5 3 1

(Traversal goes deep along one path: $0 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 1$, then backtracks)

6 Time Complexity

	Sequential DFS	Parallel DFS
Time	$O(V + E)$	$O\left(\frac{V+E}{P}\right)$ ideally
Space	$O(V)$	$O(V)$

Where V = Vertices, E = Edges, P = Number of Threads.

7 Compilation

Listing 2: Compile and Run

```
g++ -fopenmp DFS.cpp -o DFS
./DFS
```

Note: The `-fopenmp` flag is mandatory to enable OpenMP directives.

8 Conclusion

In this practical, we implemented Parallel Depth First Search on an undirected graph using OpenMP. The `#pragma omp parallel for` directive was used to explore adjacent nodes of each vertex in parallel across multiple threads. This demonstrates how parallelism can improve the performance of graph traversal algorithms on multi-core processors. The visited array prevents revisiting nodes, and the stack-based iterative approach ensures systematic depth-first exploration of the graph.